

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals and Data Structures	Course Code: CMPE-112L
Assignment Type: Lab Report	Dated: 16th April, 2026
Semester: 1st	Session: 2026
Lab/Project/Assignment #: 10	CLOs to be covered: CLO2
Lab Title: Recursion	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO2	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Mark s	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 10: Recursion

Lab Goals/Objectives:

To understand the concept of recursion, where a function calls itself, and to apply it to solve problems such as computing a factorial and generating a Fibonacci sequence.

Equipment Required:

Python 3, VS Code / PyCharm

Theory:

Recursion is a technique in which a function calls itself to solve smaller instances of the same problem. Every recursive function must have a base case to stop the recursion; otherwise it will run indefinitely and raise a maximum recursion depth error.

Lab Work:**Task 1: Factorial using Recursion**

```
def fact(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n* fact(n-1)  
print(fact(1))  
print(fact(5))  
print(fact(6))
```

Result:

```
1  
120  
720  
[0, 1, 1, 2, 3]
```

Task 2: Recursion without a Base Case

```
# def fact(n):  
#     return n*fact(n-1)  
# print(fact(5)) # will show error as maximum recursion depth is reached
```

Result:

```
[Previous line repeated 996 more times]  
RecursionError: maximum recursion depth exceeded
```

Task 3: Fibonacci Sequence with Recursion

```
def fibonacci(n):  
    if n<=0:  
        return []  
    elif n == 1:  
        return [0]  
    elif n==2:  
        return [0, 1]  
    else:  
        seq = fibonacci(n-1)  
        seq.append(seq[-1]+seq[-2])  
        return seq  
print(fibonacci(5))
```

Result:

```
[0, 1, 1, 2, 3]
```

Conclusions:

Recursion was applied to compute factorials and generate a Fibonacci sequence, and the importance of a base case in preventing infinite recursion was observed.